

RIME HACKATHON · PROJECT BLUEPRINT

Continuum

A voice-native conversation continuity assistant — whispered recaps that catch you up before you ever say “hello.”

Prepared for: Build in Google Antigravity (agentic development platform)

Primary voice provider: Rime (Coda / Mist v2 / Mist v3, via LiveKit Agents)

Hackathon paths targeted: Interruption & Recovery · Conversation Continuity During Tool Work

Document purpose: End-to-end guidance — problem, architecture, modules, tech stack, build plan, and rubric-aligned submission checklist

Contents

- 01 Executive Summary
- 02 Hackathon Rubric Alignment
- 03 The User & The Problem
- 04 Core Product Concept
- 05 Scope: MVP vs. Stretch
- 06 System Architecture
- 07 Module Breakdown
- 08 Team Split, Repo Structure & Git Workflow
- 09 Key Scenarios & Sequence Flows
- 10 Technology Stack
- 11 Building This in Google Antigravity
- 12 Rime Integration Details
- 13 Acceptance Tests & Stress Cases
- 14 Privacy, Consent & Safety
- 15 Submission Checklist (Rules Compliance)
- 16 Suggested Build Timeline
- 17 Risks & Mitigations
- 18 Resource Appendix

01 · Executive Summary

Continuum is a voice assistant for people who get pulled off one conversation by a higher-priority one and lose the thread. When the interrupted caller reconnects — minutes later or the next day — Continuum privately whispers a fast, accurate recap into the user's ear *before they even answer*, so the person on the other end never experiences a delay, a repeated question, or an awkward "sorry, where were we?"

The project is built around one hard, provable voice problem: **deliver a spoken recap with near-zero added latency to the live call, on a private channel the other party never hears, with information that is fact-checked for staleness**. That is a direct match for two of the hackathon's named hard-problem paths: **Interruption and recovery**, and **Conversation continuity during tool work**. Rime is not a bolt-on — it is the mechanism that makes the whole idea viable, because the recap must be spoken, fast, low-latency, and understandable at a glance, while the user's eyes and hands are occupied re-engaging with a live conversation.

This document gives you: the sharpened product concept, a modular architecture, a concrete tech stack, a plan for building it efficiently inside **Google Antigravity** using parallel agents, the exact Rime features to use for each module, a set of acceptance tests that satisfy the "evidence and reproducibility" rubric criterion, and a submission checklist derived directly from the hackathon rules.

02 · Hackathon Rubric Alignment

Every design decision in this document is traceable back to a scored rubric line. Keep this table pinned while you build — if a feature doesn't map to a row below, it's a distraction from judging weight.

Rubric Criterion	Weight	How Continuum Addresses It
Problem & necessity of voice	25%	Recap must be <i>heard</i> , not read, because the user's eyes/hands are committed to a live conversation. Removing speech collapses the product back into "read your notes app," which defeats the purpose (recap has to land before/while a call is live).
Hard voice engineering	25%	Solves interruption/recovery: detects reconnect, fences stale audio/context, delivers a whispered recap on a private channel with zero added delay to the other party, and re-verifies time-sensitive facts before speaking them.
Evidence & reproducibility	20%	A scripted, repeatable stress test (Section 12) with before/after clips, timing measurements, and a fixture that forcibly drops and re-establishes a call with a changed backend fact.
Rime integration & voice experience	20%	Rime is the only spoken output. Model/voice/speed/prompting choices are deliberate and testable: <code>timeScaleFactor / speedAlpha</code> for pacing, <code>inlineSpeedAlpha</code> to slow down critical facts, <code>spell()</code> for IDs/names, a distinct persistent voice identity for the assistant.
Demo clarity	10%	A single 4-5 min demo: normal call → forced disconnect mid-sentence → reconnect next day → whispered recap plays during ring → stale fact is caught and flagged → user answers already caught up.

Eligibility reminders baked into this design: Rime is the *primary* spoken output (not a welcome message); the repository will contain a working, reproducible path; no credentials are ever hard-coded; fallback behavior (if any) is disclosed and Rime remains default in the judged flow.

03 · The User & The Problem

| Primary user

A working professional who juggles concurrent, high-stakes conversations and phone calls where being interrupted is routine and costly: sales reps, account managers, doctors on call, support/dispatch leads, lawyers, ops managers, recruiters. The common trait: they get pulled from Conversation A into higher-priority Conversation B, and A or B later resumes after a gap of minutes to a full day.

| The problem, precisely

- **Task-switch cost is real and expensive.** Reconstructing "where were we" wastes time and signals disorganization to the other party.
- **The naive fix breaks the experience.** Asking an assistant out loud to "catch me up" after answering makes the other person sit through dead air — this is the exact failure mode a judge will stress-test.
- **Facts go stale.** A quote, an inventory count, a ticket status discussed yesterday may have changed by the time the call resumes.
- **Privacy matters.** The recap is for the user only; it must never leak into what the other party hears.

04 · Core Product Concept

| The key mechanic: recap before pickup, not after

When a saved-thread contact calls back, Continuum recognizes the caller (caller ID / SIP header match against a stored thread) and uses the **ringing window** — typically 15–25 seconds across a few rings — to privately speak a recap into the user's earpiece *while the phone is still ringing on the caller's end*. The caller hears normal ringback tone the whole time. By the time the user answers, they are already caught up. This adds **zero** latency to the caller's experience because the recap happens entirely before the call is connected.

Recap structure — headline first, expandable

1. **TL;DR (1 sentence, < 3s to speak):** "Z wanted the Q3 number; you said you'd check with finance."
2. **Key details (if ring window allows):** open commitments, numbers, names, next step.
3. **Freshness flag (if applicable):** "Heads up — the price you quoted has changed since you last spoke."

| Fallback: live whisper channel (if answered before recap finishes, or no ring buffer)

A second, private audio track is delivered only to the user's device (earpiece/AirPods), entirely separate from the leg of the call the other party hears. The user can silently query it mid-call — "remind me what he wanted" — and hear a whispered answer with zero interruption to the live audio the other party hears. This is the same mechanism used in call-coaching software, repurposed for personal memory instead of coaching prompts.

| Fact-freshness check

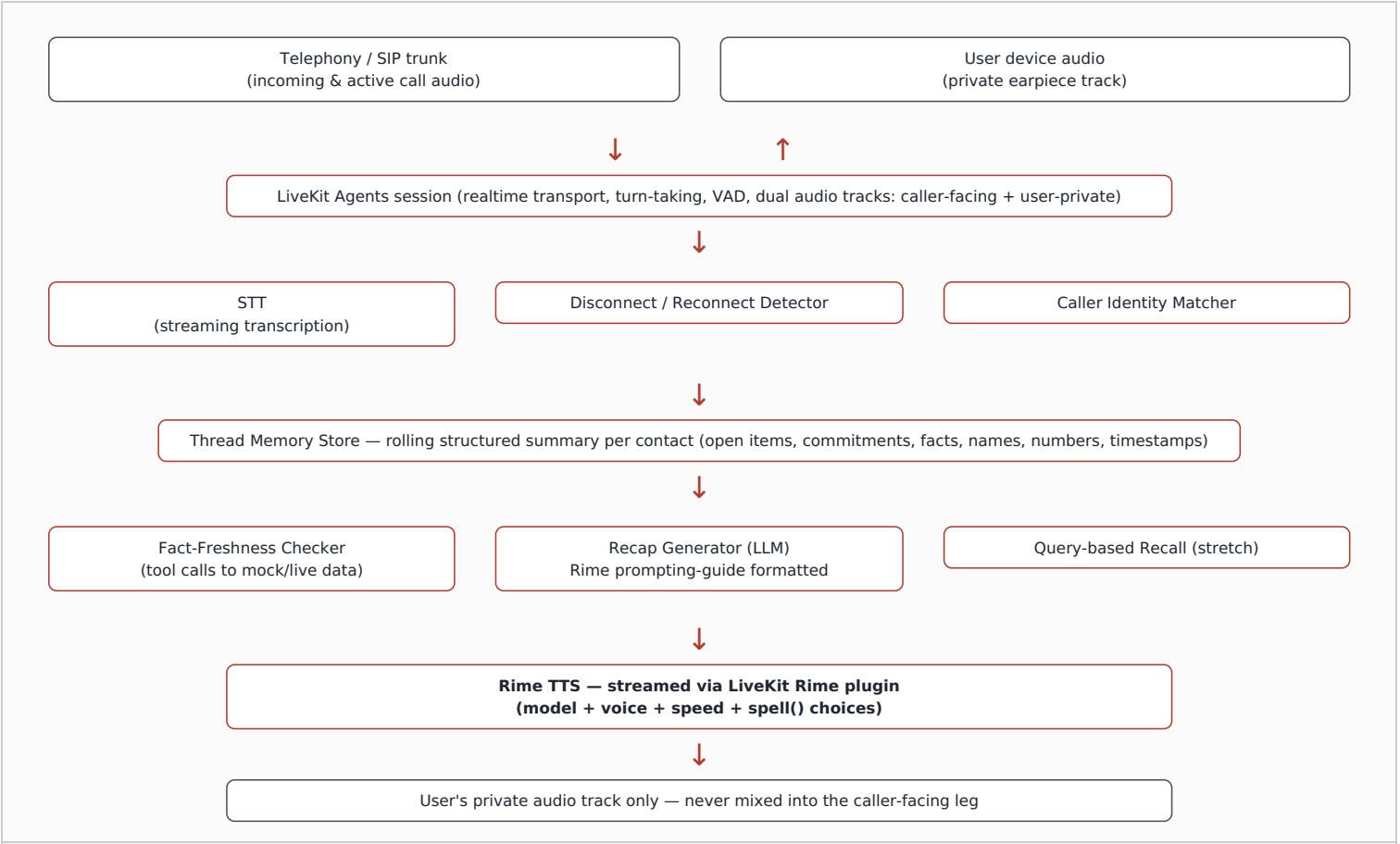
Before speaking any time-sensitive detail (price, availability, status, deadline), the recap generator silently calls the relevant backend/tool to re-verify it, and explicitly flags any discrepancy rather than repeating stale information. This directly demonstrates the "conversation continuity during tool work" rubric path.

05 · Scope: MVP vs. Stretch

Hackathon judging rewards one convincingly solved hard problem over a broad, shallow assistant. Scope the demo to a single phone thread (User ↔ Contact Z). Treat the original in-person conversation (User ↔ Y) as a documented extension, not a build target.

	Capability	Notes
MVP	Live call capture & rolling transcript/summary for one active phone thread	Speech-to-text + running structured memory (open items, facts, numbers, names)
MVP	Disconnect detection + thread persistence	Store the "last state" of the conversation keyed to caller identity
MVP	Reconnect detection via caller ID / SIP match	Recognize this is a resumed thread before answering
MVP	Pre-answer whispered recap during ring window	The core "hard voice problem" — must be measured for latency
MVP	Private live whisper fallback channel	For instant-connect scenarios with no ring buffer
MVP	Fact-freshness check on one mock data source	e.g., a mock "price/ticket status" API you control for the demo
Stretch	Second thread (User ↔ Y, in-person)	Local mic capture + context isolation between two threads
Stretch	Multilingual recap (code-switched)	If Z sometimes speaks a different language than the recap
Stretch	Query-based recall ("did we agree on the budget?")	Turns recap into an on-demand Q&A over the transcript

06 · System Architecture



Design invariant that judges will stress-test

The caller-facing audio path and the user-private audio path must be architecturally separate tracks in the same LiveKit room, not a single mixed track with volume tricks. This is what guarantees the recap can never leak to the other party, and it's the cleanest way to prove "fencing" of the assistant's output from the live call.

07 · Module Breakdown

Module	Responsibility	Key Tech
Call Session Manager	Owns the LiveKit room lifecycle: joins the caller-facing leg, opens a second private track for the user, tracks call state (ringing / connected / disconnected).	LiveKit Agents, SIP bridge (LiveKit telephony / Twilio SIP trunk)
Streaming STT	Transcribes both legs in real time into a shared rolling buffer, tagged with speaker and timestamp.	Deepgram / AssemblyAI / LiveKit-supported STT plugin
Disconnect / Reconnect Detector	Flags an abrupt call end (vs. a normal hangup with closing words) and, on any new inbound call, checks whether the caller ID matches a paused thread.	Simple heuristic (call duration + absence of closing turn) + caller ID / SIP header lookup
Thread Memory Store	Persists a structured, continuously-updated summary per contact: open asks, commitments made, numbers/names/dates mentioned, last-updated timestamp. Not a raw transcript dump.	Postgres / SQLite + a lightweight schema; embeddings optional for query-based recall (stretch)
Fact-Freshness Checker	Before the recap is spoken, re-verifies any time-sensitive fact against a live/mock data source and produces a diff if something changed.	A small internal REST endpoint you control for the demo (e.g., a mock pricing/ticket API)
Recap Generator	Turns the structured summary + freshness diff into short, spoken-style text: TL;DR first, then expandable detail, formatted per Rime's prompting guide (short sentences, natural disfluencies, no SSML).	LLM (Gemini or Claude via Antigravity's model access) with a system prompt derived from Rime's drop-in voice prompt
Rime TTS Delivery	Synthesizes and streams the recap. Chooses model (Coda for expressive natural delivery, Mist v2/v3 if per-word speed control on critical facts is needed), sets <code>timeScaleFactor / inlineSpeedAlpha</code> , wraps IDs/names in <code>spell()</code> .	Rime API via LiveKit Rime plugin; regional endpoint nearest to deployment
Private Whisper Channel	Routes recap audio (and any live mid-call query response) only to the user's device track; never mixed into the caller-facing track.	LiveKit multi-track room configuration
Consent & Privacy Layer	Manages recording disclosure, redaction of sensitive fields, and de-identifies any stored data used in demos.	Policy layer + config flags; synthetic data for demo recordings
Evaluation Harness	Scripts that force a disconnect, force a reconnect, inject a changed backend fact, and measure latency + accuracy automatically for the RIME_EVIDENCE.md artifact.	Python test scripts + saved audio clips + a results log

08 · Team Split, Repo Structure & Git Workflow

The 10 modules in Section 07 form a pipeline (audio → transcript → memory → recap text → speech), not independent silos — so a clean 4-person split means grouping modules by who talks to whom, then fixing the data contracts between groups on day one so everyone can build in parallel without blocking each other. Consent/Privacy and the Evaluation Harness are cross-cutting and touched by everyone, not owned by one pair.

The 4 pairings (2 modules each)

Pair	Owns	Why paired together
A — Transport	Call Session Manager + Private Whisper Channel	Both are the same LiveKit room configuration — one caller-facing track, one private track. Building one means you're already deep in the other.
B — Signal	Streaming STT + Disconnect/Reconnect Detector	Both consume raw call audio/state and emit events (transcript chunks, call-state changes) into the pipeline.
C — Brain	Thread Memory Store + Recap Generator	Memory schema in, recap text out — one continuous "what do we know, what do we say" responsibility.
D — Voice & Facts	Fact-Freshness Checker + Rime TTS Delivery	Both sit immediately before speech: verify the fact, then speak it (with <code>spell()</code> /speed tuning applied).

Data flows A → B → C → D → A (the recap eventually plays back on A's private track). The fix for this coupling isn't pretending the pairs are independent — it's agreeing on every message that crosses a pair boundary *before* anyone writes implementation code, and having every pair build against a mock of their neighbor's output.

Step 1 (do this together, all 4 people, before splitting up): lock the shared contracts

Define the shape of every message that crosses a pair boundary as an actual typed schema (Pydantic for Python, Zod for TypeScript), not a verbal agreement:

```
/shared/schemas/  
transcript_event.py      # Signal -> Brain  
call_state_event.py      # Transport -> Signal  
thread_summary.py        # Brain's internal + output state  
recap_request.py         # Brain -> Voice&Facts  
freshness_result.py      # internal to Voice&Facts  
tts_request.py           # Voice&Facts -> Transport (private track)
```

This schema set is the single most important artifact of the whole build — once it exists and is agreed, all 4 pairs can work in true parallel.

Step 2 — Repository structure

```
/continuum  
/shared/schemas/        # the contracts above -- owned by everyone, reviewed by all 4  
/modules/transport/      # Pair A  
/modules/signal/         # Pair B  
/modules/brain/          # Pair C  
/modules/voice/          # Pair D  
/mocks/                  # fake implementations of each module's output, for others to build against  
/evaluation/              # shared cross-cutting: the acceptance tests from Section 13  
/docs/ARCHITECTURE.md    # this document, condensed  
CODEOWNERS
```

Each module folder ships two implementations behind one interface: a **mock** version (returns canned/fake data instantly) and the **real** version. Example: Pair D can build Rime TTS against Pair C's `mock_recap_generator`, which just returns a fixed string, without waiting for Brain to be finished. Toggle with an environment flag such as `USE MOCKS=true`.

Step 3 — Git workflow

- main** is protected: merges only via pull request, and only once CI passes.
- One long-lived branch per pair — `team/transport`, `team/signal`, `team/brain`, `team/voice`. Each pair commits to their own branch and opens small PRs into `main` daily, not once at the end, so integration issues surface early instead of on the last day.
- A `CODEOWNERS` file auto-requests the right reviewers on every PR:

```
/modules/transport/ @person1 @person2  
/modules/signal/    @person1 @person2 # or whichever 2 of your 4 usernames own this pair  
/modules/brain/     @person3 @person4  
/modules/voice/     @person3 @person4  
/shared/schemas/   @person1 @person2 @person3 @person4 # everyone reviews contract changes
```

Since you're 4 people covering 4 pairs (not 8 people), each person's GitHub username goes on their pair's two module folders. The one rule worth enforcing strictly: **no solo merges to** `/shared/schemas/` — a silent change there breaks every other pair's mocks without warning, so it always needs a quick heads-up in the group chat before merging, even though it's a small file.

Step 4 — Integration checkpoints

Don't wait until the end to plug everything together. Set two fixed checkpoints against the build timeline in Section 16:

- **End of Day 2:** each pair demos their module running against the other pairs' mocks — this proves the shared contracts actually hold up, before any real implementation is finished.
- **End of Day 4:** swap every mock for its real implementation and run the full pipeline once, live, end to end.

This way, a delay in one pair (historically Transport's SIP/telephony setup is the fiddliest part) never blocks the other three pairs from having working, testable code the whole time.

09 · Key Scenarios & Sequence Flows

Scenario A — Normal call, no interruption

1. User and Z connect via LiveKit/SIP.
2. STT streams transcript; Thread Memory Store updates a rolling structured summary in the background.
3. Call ends normally — no recap needed next time (or a very short "last touchpoint" note only).

Scenario B — Disconnect mid-call, reconnect within minutes

1. Call drops abruptly (no closing turn detected). Session Manager marks the thread "interrupted," timestamps the last spoken turn.
2. Z calls back. Caller Identity Matcher recognizes the number against the paused thread *before* the call is answered.
3. During ringback, Recap Generator produces a short TL;DR from the last few turns; Rime speaks it into the user's private track only.
4. User answers already oriented; Z experiences only normal ring-then-answer.

Scenario C — Reconnect the next day, with a stale fact

1. Thread Memory Store holds yesterday's structured summary, including a quoted price of \$400.
2. Z calls back next day. Caller match found; ringback window opens.
3. Fact-Freshness Checker calls the mock pricing API in parallel with recap generation and finds the price is now \$420.
4. Recap Generator inserts the flag: "Heads up — you'd quoted \$400, it's actually \$420 as of this morning."
5. Rime speaks the recap (fast pacing on connective text, `inlineSpeedAlpha` slowed on the number) before the user answers.

Scenario D — Instant connect, no ring buffer (fallback)

1. Call connects immediately with no usable ring window.
2. User answers normally; Continuum stays silent on the caller-facing track.
3. User silently invokes the assistant (button press, wake phrase into the private mic path, or a quiet tap gesture) and asks "remind me what he wanted."
4. Rime whispers the answer on the private track only, with zero audible effect on the live call.

10 · Technology Stack

Layer	Recommended	Why
Realtime transport & orchestration	LiveKit Agents (open source) + LiveKit Cloud or self-hosted	Handles turn-taking, VAD, multi-track rooms (needed for the private whisper track), and has an official Rime plugin.
Telephony bridge	LiveKit SIP / Twilio SIP trunk into LiveKit	Lets a real phone number ring into your agent, giving you a genuine ring-window to work with rather than a browser-only demo.
Speech-to-text	Deepgram or AssemblyAI streaming STT (LiveKit-supported plugin)	Low-latency streaming transcription for both legs of the call.
Text-to-speech	Rime — Coda (expressive default) and/or Mist v2/v3 (per-word <code>inlineSpeedAlpha</code> , <code>spell()</code>)	Primary spoken output per hackathon rules; only Rime should be the judged voice path.
LLM (recap + reasoning)	Gemini 3 family or Claude, accessed through Antigravity's built-in multi-model support	Antigravity lets you swap models per-agent; use a fast model for the TL;DR path and a stronger one for the freshness-diff reasoning.
Memory / structured store	Postgres (or SQLite for hackathon speed) + a simple JSON schema per thread	You need structured facts (open items, numbers, names, timestamps), not a raw transcript blob, to generate a short recap.
Mock "live data" source for staleness demo	A tiny FastAPI/Flask service you control, seeded with a changeable value	Lets you deterministically demonstrate the fact-freshness stress case.
Frontend / control surface	Minimal web dashboard (thread list, call log, latency graphs) — React	Only needed to make the demo legible to judges; the product itself is voice-first.
Evaluation & logging	Python scripts, saved WAV/MP3 clips, a results table	Feeds directly into RIME_EVIDENCE.md.

11 · Building This in Google Antigravity

Antigravity's value here is running several of the modules above as **parallel agents** instead of building everything sequentially yourself. Structure the work this way:

Project setup

- Start from a plain LiveKit Agents scaffold checked into the repo; add an `AGENTS.md` at the project root describing the architecture in Section 06, the module boundaries in Section 07, and the "never mix the private track into the caller-facing track" invariant as a hard rule the agents must not violate.
- Use the **Editor view** for the LiveKit/SIP plumbing and audio-track wiring — this is fiddly, synchronous, and benefits from tab-completion and inline fixes.
- Use the **Manager view** to spawn separate agents for modules that don't depend on each other's internals, for example:
 1. Agent 1 — Thread Memory Store schema + persistence layer
 2. Agent 2 — Recap Generator prompt engineering (seeded with Rime's drop-in voice system prompt from Section 11)
 3. Agent 3 — Mock fact-freshness API + the deliberate "changed value overnight" fixture
 4. Agent 4 — Evaluation harness scripts (force-disconnect, force-reconnect, timing capture)
 5. Agent 5 — Minimal dashboard for demo legibility
- Use Antigravity's built-in Chrome browser control to visually verify the dashboard and to drive any browser-based test call flow end to end, screenshotting each state for your README/demo evidence.
- Because Antigravity supports multiple models per agent, assign a fast/cheap model to latency-sensitive paths (TL;DR generation) and a stronger reasoning model to the freshness-diff logic, then compare in your evidence doc.

Guardrails to write into AGENTS.md

- Rime must remain the only TTS provider in the judged path; agents must not silently substitute a placeholder browser `speechSynthesis` call "to save time" — flag it instead if Rime access is unavailable.
- Never commit API keys; use server-side secrets/environment variables only, and keep a `.env.example` with placeholders.
- Any recap text must pass through the Rime prompting-guide formatting step (short sentences, no SSML, `spell()` for IDs) before being sent to synthesis — make this a lint-style check, not a suggestion.

12 · Rime Integration Details

Model choice

Use case	Model	Reasoning
Default recap delivery	Coda	Natural, expressive default; good baseline for the "sounds like a person" requirement.
Recap with a critical number/name that must land clearly	Mist v2 or v3	Only the Mist family supports per-word <code>inlineSpeedAlpha</code> , letting you slow down just the important token.
Reading confirmation codes / ticket IDs / account numbers	Mist family + <code>spell()</code>	<code>spell()</code> is processed by Mist (grouped in threes on v3); Coda passes it through unprocessed, so use Mist when this matters.

Speed control (docs.rime.ai/docs/speed)

- **Coda / Mist v3:** use `timeScaleFactor` — below 1.0 speeds up, above 1.0 slows down. Suggest ~0.85 for the connective filler of the recap.
- **Mist v2:** use `speedAlpha` — lower than 1.0 is faster.
- **Per-word slow-down on Mist v2/v3:** use `inlineSpeedAlpha` with bracketed words, e.g. "Heads up, the price is now [four hundred twenty dollars]" with a value above 1.0 on that phrase to slow it relative to the rest.
- Always evaluate near 1.0 first and move incrementally — large jumps reduce naturalness, which is itself a rubric-relevant detail (voice experience quality).

Prompting (docs.rime.ai/docs/prompting) — adapted system prompt for the Recap Generator

```
You are generating a SPOKEN recap that will be read aloud by Rime.
Write for the ear.

1. Lead with the one-sentence headline. No preamble.
2. Keep every sentence under 20 words.
3. Use natural disfluencies sparingly ("so," "yeah," "one sec") —
   never stack them.
4. Use punctuation as your only prosody tool. No SSML, no tags.
5. If a fact has changed since it was last discussed, say so plainly
   and early: "Heads up -- [old value] is now [new value]."
6. Wrap any ID, ticket number, or code in spell(...).
7. Never invent details not present in the thread memory. If something
   is uncertain, say "I'm not fully sure, but..." rather than guessing.
8. End with the single most useful next action, if one exists.
```

Other relevant docs to keep open while building

- **Regional endpoints** — pick the endpoint closest to your LiveKit deployment region to minimize the network leg of your latency budget.
- **Voices & the live catalog** (`users.rime.ai/data/voices/all-v2.json`) — query the current catalog at build time rather than hard-coding a voice ID list, since the hackathon rules require using the live catalog at submission time, not a stale copy.
- **Writing for the ear** guide — use it to review your recap script line by line before recording your final demo clips.

13 · Acceptance Tests & Stress Cases

Define these before you touch the demo recording, per the hackathon rules. Each row should produce a saved artifact (clip, log, or screenshot) for RIME_EVIDENCE.md.

Test	Procedure	What "pass" looks like
Pre-answer latency	Force-disconnect a live call mid-sentence; force Z to call back; measure time from first ring to first audible recap word.	Recap begins within the first 1–2 rings, well before a human would typically answer.
Zero impact on caller	Record the audio Z actually hears throughout reconnect + recap window.	Z's audio is indistinguishable from an ordinary ringback-then-answer call; no recap audio present.
Context fencing	Run two simulated threads (Z and a second contact) in parallel; verify recap for Z never includes content from the other thread.	Zero cross-thread leakage in transcript review.
Freshness catch	Change the mock backend value between the original call and the reconnect; verify the recap explicitly flags the discrepancy rather than repeating the stale figure.	Spoken recap states both the old and new value correctly.
Interruptibility	User barges in mid-recap ("skip, I remember").	Queued Rime audio stops promptly; no orphaned audio continues playing.
Fallback whisper channel	Simulate an instant-connect call with no ring buffer; invoke the live whisper mid-call.	Whisper is audible only on the user's private track, with no audible artifact on the caller-facing track.
Pronunciation check	Run any name, ID, or number that appears in the demo through <code>/textnorm</code> before final recording.	Confirmed correct reading; documented before/after if a fix was needed.

14 · Privacy, Consent & Safety

- The system records and stores conversations involving a third party (Z, and Y in the stretch scope) without their explicit knowledge in a live deployment — disclose this plainly as a known limitation in your README, and describe what a production version would need (e.g., a recorded-line disclosure, opt-out mechanism, retention limits).
- For the hackathon demo itself, use synthetic or de-identified call scripts and voices rather than a real colleague's actual conversation content.
- Give the Thread Memory Store a redaction option for sensitive fields (financial figures, health information, personal identifiers) before anything is persisted beyond the active session.
- Keep the private whisper track genuinely private at the transport layer (separate LiveKit track/permission), not just muted client-side, so there's no path by which it could leak to the recorded call.

15 · Submission Checklist (Rules Compliance)

- ☐ **Demo** — 4–5 minute recording showing: the user/problem, the normal flow, the chosen hard voice problem (pre-answer whispered recap), one deliberate stress case (freshness catch or forced disconnect), the measured result, and which speech provider is active on screen.
- ☐ **Working repository** — all demonstrated behavior must exist in the code judges can inspect and reproduce.
- ☐ **README** — setup instructions, architecture diagram (Section 06), third-party services used, known limitations, failure behavior, and the exact Rime model ID, speaker, language, endpoint, audio format, and transport used.
- ☐ **RIME_EVIDENCE.md** — the hard voice claim, the acceptance test (Section 12), procedure, result, limitations, and a repeatable script/fixture.
- ☐ **.env.example** — placeholders only, no live credentials anywhere in code, docs, screenshots, or recordings.
- ☐ **Preflight** — run the organizer-provided Rime configuration/secret preflight check before submitting; confirm the model/voice/language combination is currently valid in the live catalog.
- ☐ **Fallback disclosure** — if you build any non-Rime fallback for resilience, make the active provider visibly observable in the demo, and keep Rime as the default path judges see.
- ☐ **No incidental-only Rime usage** — confirm Rime is not limited to a welcome message or final confirmation; it must carry the core recap experience.

16 · Suggested Build Timeline

Phase	Focus
Day 1	LiveKit Agents scaffold, SIP bridge working end-to-end (a real call can ring in and connect); basic streaming STT wired up. Write AGENTS.md.
Day 2	Thread Memory Store schema + persistence; Disconnect/Reconnect Detector; caller identity matching. Spawn parallel Antigravity agents for the mock freshness API and the evaluation harness scripts.
Day 3	Recap Generator prompt (Section 11) + Rime TTS wiring through the LiveKit plugin; private whisper track configuration; first end-to-end pre-answer recap.
Day 4	Run all acceptance tests (Section 12); tune speed/voice/spell() choices based on results; fix any context-fencing or latency issues found.
Day 5	Record demo, write README + RIME_EVIDENCE.md, run the secret/config preflight, final review against the Section 14 checklist.

17 · Risks & Mitigations

Risk	Mitigation
Ring window too short to speak a useful recap	Front-load the single most useful sentence first (headline-first design already handles this); measure actual ring durations on your test SIP setup early.
Real telephony/SIP setup is time-consuming	Get the SIP bridge working on Day 1 before building anything else; if it slips, fall back to a browser-simulated "ring" state for the demo but disclose this clearly as a limitation.
Private track accidentally leaks to caller	Treat this as the #1 thing to test; write an automated check that inspects the caller-facing track's audio for any recap content.
Recap sounds robotic / hard to parse quickly	Follow the Rime prompting guide closely; test multiple phrasings and speeds against real listeners before locking the final script.
Freshness check adds latency	Run the freshness lookup in parallel with recap-text generation, not sequentially before it; design the mock API to respond in well under a second.

18 · Resource Appendix

Resource	Use
Writing for the ear: prompting your TTS to sound human	Recap script style guide
Voice AI with Rime project catalog (GitHub)	Inspiration; confirm your idea extends rather than duplicates existing entries
TTS in five minutes (quickstart)	Fastest path to a first working Rime call
Models	Coda vs. Mist v2/v3 tradeoffs
Voices and supported languages / live catalog JSON	Select and verify voice/language combination at build time
Regional endpoints	Latency-minimizing endpoint selection
Prompting guide + drop-in system prompt	Base for the Recap Generator's system prompt
LiveKit integration docs	Realtime transport, dual-track audio, telephony bridging
Qwen Audio Agent (optional)	Reference full-duplex frontend if you need one beyond LiveKit Agents
Playback speed docs	<code>timeScaleFactor</code> , <code>speedAlpha</code> , <code>inlineSpeedAlpha</code> reference

Prepared as a working blueprint for a Rime hackathon submission. Treat every number and mechanism above as a starting hypothesis to test, not a guarantee — the acceptance tests in Section 12 are what actually earn the "evidence and reproducibility" score.